CONTENTS

Advanced | Information retrieval | Condensed | IBM Technology

From Inverted Indices to Agentic RAG: How Retrieval Grew Up

Follow information retrieval from keyword matching to systems that decide for themselves what to look at.

For engineers building search or RAG systems who want to know which stage of the evolution their system is stuck at · 26 July 2026

Every section, key point, term and question from the full lesson is here. The extended explanations and the additional examples are not.

Watch the source video | Read the full lesson | Read the highlights

BEFORE YOU START

- Familiarity with the idea of a search index
- Comfort reading Python
- A basic idea of what an embedding is, though the lesson builds it up

Overview

Search engines did not get smarter overnight. Information retrieval grew up one step at a time, and each step was a response to a specific, nameable failure of the step before it. Keyword indices could not handle synonyms. Semantic search could not handle exact identifiers. Static RAG pipelines could not adapt when one retrieval was not enough.

That progression is worth knowing in detail, because almost every retrieval system in production today is sitting at one of these stages, and its problems are the documented problems of that stage. A team fighting recall issues with better embeddings is often at the point where hybrid retrieval was invented. A team whose RAG works on simple questions and collapses on multi-hop ones is at the point where agentic retrieval was invented.

The endpoint so far is retrieval as a tool inside a reasoning loop, rather than a fixed step ahead of one. The agent decides whether to search, where, what to ask, and when it has enough. As the video puts it: the hardest part is not generation, it is deciding what to look at.

KEY TAKEAWAYS

- ✓ An inverted index maps terms to the documents containing them, which is what makes keyword search fast.
- ✓ BM25 ranks by term frequency, inverse document frequency and document length, and remains a strong baseline decades on.
- ✓ Lexical search treats words as symbols, so synonyms, ambiguity and intent are invisible to it.
- ✓ Embeddings are learned from context, which is why 'espresso' lands near 'coffee' and far from 'house'.
- ✓ Hybrid retrieval combines lexical precision with semantic recall because each covers the other's failures.
- ✓ LLMs predict likely tokens rather than retrieving facts, so their knowledge is frozen and unattributable.
- ✓ Naive RAG is linear — embed offline, retrieve once, generate — and is only ever as good as that single retrieval.
- ✓ Agentic RAG makes retrieval a tool invoked during reasoning, so the system can iterate, verify and stop when it has enough.

01 Keyword search: inverted indices and ranking

0:00

> The earliest systems answered one question — where does this word appear — using a mapping from terms to documents, then ranked the hits by statistical importance.

KEY POINTS

- → An inverted index maps each term to the documents containing it, making lookup proportional to matches rather than corpus size.
- → TF-IDF weights a term by its frequency in the document against its rarity across the corpus.
- → BM25 adds term-frequency saturation so repetition has diminishing returns.
- → BM25 also normalises for document length, so short focused documents are not penalised.
- → Lexical ranking needs no model or training and is fully interpretable — it remains a serious baseline.

An inverted index, in full

The whole data structure. Querying 'python error' intersects two posting lists and returns their overlap. Real implementations add positions for phrase queries and skip lists for faster intersection, but this is the idea in its entirety.

```
from collections import defaultdict

index = defaultdict(set)

def add(doc_id, text):
    for term in tokenize(text.lower()):
        index[term].add(doc_id)

add(1, "Python error handling with try and except")
add(2, "Ball python care and feeding")
add(3, "Error codes in the payment API")

# index["python"] = {1, 2}
# index["error"]  = {1, 3}

def search(query):
    sets = [index[t] for t in tokenize(query.lower()) if t in index]
    return set.intersection(*sets) if sets else set()

search("python error")   # {1}
```

02 Where lexical matching breaks

1:24

> It treats words as symbols, not as meaning. Synonyms, ambiguity and intent are invisible.

KEY POINTS

- → Lexical matching compares symbols, so it has no access to meaning.
- → Synonymy causes recall failures: the right document exists and cannot be found.
- → Ambiguity causes precision failures: identical tokens, different intents.
- → The user is forced to guess the author's exact vocabulary.
- → Neither failure can be fixed by better ranking — the candidates were never retrieved.

Two failures, two causes

Worth separating, because the fixes differ. Synonym failures are about candidate generation and need semantics. Ambiguity failures are about disambiguation and often need context about the user or the session.

```
SYNONYMY (recall)
  query : "car insurance"
  doc   : "Comprehensive vehicle coverage for private owners"
  match : none. zero shared terms, and the document is the right answer.

AMBIGUITY (precision)
  query : "help python"
  doc A : "Debugging Python tracebacks"          ← wanted, if a programmer
  doc B : "Ball python humidity requirements"    ← wanted, if a snake owner
  match : both, ranked by term statistics alone.
```

03 Semantic search: meaning as geometry

2:09

> Represent text as vectors in a high-dimensional space where similar concepts sit close together, and relevance becomes distance.

KEY POINTS

- → Embeddings are vectors positioned so that semantic similarity becomes geometric proximity.
- → They are learned by neural networks from context across massive corpora.
- → Words in similar contexts end up in similar positions, independent of spelling.
- → Retrieval becomes a nearest-neighbour problem rather than a matching problem.
- → Synonymy is solved structurally: different words for one concept share a region.

The map, in three dimensions

Real embeddings have hundreds or thousands of dimensions and no individual axis means anything interpretable. But the geometry is genuinely this: similarity is a small angle, and unrelatedness is a large one.

```
coffee    [0.0, 1.0, 0.0]
espresso  [0.1, 0.9, 0.1]     ← near coffee: same contexts, same region
latte     [0.1, 0.8, 0.2]
house     [1.0, 0.0, 0.0]     ← far: nothing in common

cosine(coffee, espresso) ≈ 0.99
cosine(coffee, house)    ≈ 0.00

# no shared letters between "coffee" and "espresso" — the closeness comes
# entirely from having been used in similar surroundings during training
```

04 Hybrid retrieval: precision and recall together

2:49

> Semantic search did not replace keyword search. Systems combining both emerged because each covers the other's failures exactly.

KEY POINTS

- → Hybrid retrieval unions lexical and semantic candidates rather than choosing between them.
- → Lexical is precise on exact tokens; semantic is robust to paraphrase.
- → The two fail in opposite directions, so together they cover far more ground.
- → Combining raw scores is fragile because the scales are incomparable.
- → Reciprocal Rank Fusion combines ranks instead of scores and needs no tuning.

Reciprocal Rank Fusion

The whole algorithm. Each document scores the sum of 1/(k + rank) across the lists it appears in, so appearing reasonably high in both beats topping one and missing the other. The constant k, conventionally 60, damps the influence of the very top positions.

```
def rrf(rankings, k=60):
    scores = defaultdict(float)

    for ranking in rankings:                # one list per retriever
        for rank, doc_id in enumerate(ranking, start=1):
            scores[doc_id] += 1.0 / (k + rank)

    return sorted(scores, key=scores.get, reverse=True)


fused = rrf([
    bm25_search(query, k=50),        # exact terms, identifiers, codes
    semantic_search(query, k=50),    # paraphrase, intent
])

# no score normalisation, no weights to tune, no scale assumptions
```

05 LLMs predict — they do not retrieve

3:31

> A language model returns the most likely continuation based on learned patterns. It has no index, no sources and no notion of looking something up.

KEY POINTS

- → LLMs predict likely continuations; they do not look anything up.
- → There is no index and no source, so nothing can be genuinely cited.
- → Knowledge is frozen at the training cutoff and excludes anything private.
- → Fluency is not evidence of grounding — the same mechanism produces both.
- → The missing capability is exactly what information retrieval already provided.

Why a model invents citations

Not a special failure mode. The reference is generated by the same next-token prediction as the surrounding prose, and a plausible-looking citation is a highly probable continuation after the words 'according to'. Nothing in the process consults anything.

```
prompt : "According to the documentation, the retry limit defaults to"

#   " 3"   0.41   plausible
#   " 5"   0.28   plausible
#   " 10"  0.12   plausible

# the model emits one. no configuration file was consulted, and the
# confidence of the output says nothing about whether it is right.
```

06 Naive RAG: the linear pipeline

4:20

> Search for relevant documents, augment the prompt with them, generate the answer. Simple, effective, and only as good as one retrieval.

KEY POINTS

- → The pipeline is: retrieve from a knowledge base, augment the prompt, generate.
- → This gives the model external memory, real citations and domain adaptation without retraining.
- → Hallucinations drop substantially because claims are grounded in supplied text.
- → The pipeline is linear: retrieval happens exactly once, with no feedback.
- → Answer quality is capped by the quality of that single retrieval.

Naive RAG in full

Genuinely the whole thing. Everything covered from here on is an attempt to fix a limitation of these five lines.

```
def naive_rag(question, store, k=5):
    hits    = store.search(embed(question), k=k)      # once
    context = "\n\n".join(h.text for h in hits)

    return model.generate(
        f"Answer from the context only.\n\n{context}\n\nQuestion: {question}"
    )
```

07 Advanced RAG: rerankers, query rewriting, hybrid

5:44

> Within a short period the simple concept accumulated real machinery — but the pipeline stayed fixed.

KEY POINTS

- → Rerankers reorder a retrieved shortlist using a more accurate, slower model.
- → Bi-encoders embed query and document separately; cross-encoders score them jointly.
- → Retrieve wide and cheap, then rerank narrow and expensive.
- → Query rewriting and expansion improve recall for short or conversational questions.
- → The pipeline was still fixed — identical steps for every query, however simple or complex.

Retrieve wide, rerank narrow

Usually the single largest accuracy improvement available to a RAG system, and it needs no change to the index. The recall you get from k=50 with the precision you get from a cross-encoder.

```
def retrieve_and_rerank(question, store, wide=50, keep=5):
    candidates = store.search(embed(question), k=wide)     # cheap, high recall

    scored = cross_encoder.predict([                       # expensive, precise
        (question, c.text) for c in candidates
    ])

    ranked = sorted(zip(scored, candidates), reverse=True)
    return [c for _, c in ranked[:keep]]

# bi-encoder  : embeds separately → index precomputable, less accurate
# cross-encoder: reads both together → far more accurate, cannot be precomputed
```

08 Agentic RAG: retrieval as a tool inside reasoning

6:29

> Retrieval stops being a fixed step before the answer and becomes something the system invokes, repeats and stops as part of thinking.

KEY POINTS

- → Agents combine an LLM with tools and decide autonomously which to invoke.
- → The agent decides whether to retrieve, where, what to ask, and when to stop.
- → Retrieval becomes a tool inside reasoning rather than a fixed preceding step.
- → This enables multi-hop research, cross-document synthesis and self-correction.
- → Latency and cost become variable; evaluation shifts from one retrieval to a whole trajectory.
- → Retrieval budgets and stopping criteria are required — an agent will otherwise search forever.

Agentic retrieval performing a multi-hop lookup

The same question naive RAG could not answer. The agent runs the first hop, reads the result, forms a second query that depends on it, and only then answers. No pipeline can do this, because the second query does not exist until the first one returns.

```
question: "Does the vendor of our payment gateway have SOC 2 certification?"

Thought: I need the vendor's name first. Search internal docs.
Action:  search_internal("payment gateway vendor contract")
Obs:     "Payments are processed via Acme Pay under MSA-2024-11."

Thought: Vendor is Acme Pay. Now check certification — that is external.
Action:  search_web("Acme Pay SOC 2 Type II report")
Obs:     "Acme Pay SOC 2 Type II, audited through 2025-06-30."

Thought: Both hops resolved, and the report has an end date worth flagging.
Answer:  Yes — Acme Pay holds SOC 2 Type II covering through 30 June 2025.
```

Glossary

Inverted index

A mapping from each term to the documents containing it — the reverse of the natural document-to-terms direction, which is what makes lookup fast.

TF-IDF

Term frequency times inverse document frequency: a term matters more if it is frequent in this document and rare across the corpus.

BM25

The standard lexical ranking function, refining TF-IDF with saturating term frequency and document-length normalisation.

Lexical search

Retrieval by literal term matching. Precise on exact strings, blind to synonyms and intent.

Semantic search

Retrieval by proximity between embeddings, so documents match on meaning even when they share no words.

Embedding

A high-dimensional vector representing text, learned so that items appearing in similar contexts occupy similar positions.

Hybrid retrieval

Running lexical and semantic retrieval together and fusing the results, since each covers the other's characteristic failures.

Reciprocal Rank Fusion (RRF)

Combining several ranked lists by summing 1/(k + rank) per document, avoiding the need to make incompatible scores comparable.

Bi-encoder

An embedding model that encodes query and document separately, allowing the corpus to be indexed in advance.

Cross-encoder

A model that reads query and document together and scores the pair. More accurate than a bi-encoder and too slow to run over a full corpus.

Reranker

A second-stage scorer, usually a cross-encoder, applied to a retrieved shortlist to reorder it more accurately.

Query rewriting

Reformulating a short or conversational question into a fuller standalone query before retrieval, to improve recall.

Naive RAG

The original linear pipeline: retrieve once, augment the prompt, generate. No feedback and no second attempt.

Agentic RAG

Retrieval invoked as a tool during reasoning, so the system decides whether, where and how often to search, and when it has enough.

Multi-hop question

A question whose answer requires a second lookup that depends on the result of the first. Beyond the reach of single-shot retrieval.

Check yourself

Answer first, then open each one.

Why does BM25 make the tenth occurrence of a query term count for much less than the second?

Because relevance saturates: a document mentioning a term fifty times is not fifty times more about it, and without damping, keyword-stuffed documents would dominate the rankings. The k1 parameter controls how quickly that saturation sets in.

Semantic search solves the synonym problem completely. Why do production systems still run BM25 alongside it?

Because embeddings are weak exactly where lexical search is strong: exact identifiers. An error code, SKU or function name has arbitrary rather than contextual meaning, so it does not embed distinctively and the exact match can rank below topically similar documents. The two fail in opposite directions, which is precisely what makes combining them worthwhile.

Why does Reciprocal Rank Fusion combine ranks rather than scores?

BM25 scores are unbounded and corpus-dependent while cosine similarities sit in a fixed range, so there is no principled way to put them on a common scale, and any normalisation you invent will be fragile across corpora. Ranks are already comparable, so fusing them needs no tuning and no assumptions about either scoring scheme.

A model produces a confident citation to a document that does not exist. Why is this the expected behaviour of the mechanism rather than a malfunction?

The citation is generated by the same next-token prediction as the surrounding text — after 'according to', a plausible-looking reference is simply a high-probability continuation. Nothing in the process consults a source, because there is no index and no lookup step. Grounding has to come from outside the model.

Why can no amount of reranking or query rewriting let a naive RAG pipeline answer 'does the vendor of our payment gateway have SOC 2 certification?'

The second search cannot be formulated until the first one returns the vendor's name, and a linear pipeline retrieves exactly once. Reranking improves the ordering of results for a query you already have; it cannot produce a query that depends on an answer you have not obtained yet. Multi-hop needs a loop.

Why does an agentic retrieval system need an explicit rule for concluding that information is absent?

Because generating another rephrased query is always easier for the model than deciding nothing exists, so without a stopping criterion the agent will keep searching until its budget runs out and then produce something anyway. 'I searched and it is not there' has to be an explicitly permitted and rewarded outcome, backed by a retrieval cap and a no-new-information check.

Where to go next

- → Implement BM25 from the formula above over a few thousand of your own documents and compare it head to head against your vector search — the baseline is stronger than most teams assume.
- → Add a cross-encoder reranker over the top 50 vector results and measure the change in recall@5; this is usually the largest single accuracy gain available without touching the index.
- → Implement RRF over your lexical and semantic retrievers, then find queries where each one rescues the other and use them as regression tests.
- → Take ten questions your RAG system answers badly and classify each as a retrieval miss, a multi-hop failure, or a whole-corpus question — the mix tells you which stage of this evolution your system is stuck at.
- → Build the conversational query-rewriting step and measure recall on follow-up questions specifically, where naive embedding of pronouns and references fails hardest.
- → Instrument an agentic retrieval loop to record how many searches each question triggers, and check the distribution for questions that never terminate cleanly.

→ Read the original RAG paper (Lewis et al., 2020) and the ReAct paper (Yao et al., 2022) back to back — the gap between them is exactly the gap between the last two sections of this lesson.
